

Automated Financial Contract Auditing

A two-step pipeline: fast fact extraction with GLiNER + deep audit reasoning with Qwen2.5-7B

Author: Kushagra Singh Kushwah | Project: IIm-financial-analyst-agent | Comprehensive Briefing

1. The Problem with Using Only a Large Language Model

My goal in this project was to build an automated auditor that reviews commercial contracts, spots late penalty fees, and calculates financial risk. At first, I tried doing everything with a single large model: **Qwen2.5-7B**, trained using reinforcement learning (GRPO).

While Qwen is great at reasoning once it has the facts, asking a huge 7-billion parameter generative model to read through entire contracts to pull out exact numbers led to three major problems:

- **It can change or round numbers:** Generative LLMs sometimes paraphrase '2.5% per week of delay' into '2%' or round dollar amounts. In a financial audit, even a small change in numbers is a serious error.
- **No exact text positions:** Generative models produce plain text, but they cannot tell you the exact character location (like indices [start: 207, end: 237]) in the original document. Without exact positions, users cannot click on an alert in a web dashboard and jump straight to the highlighted clause in the original contract.
- **Slow and expensive:** Sending 15 to 20 pages of legal text into a 7B model uses thousands of tokens and takes several seconds per document.

To fix this, I split the work into two steps: a small, ultra-fast model (**GLiNER**) pulls out the exact facts first, and then Qwen reasons over those facts.

2. What is GLiNER and Why It Works So Well

GLiNER stands for *Generalist and Lightweight Model for Named Entity Recognition*. It is a compact neural network built on DeBERTa-v3 that runs on a normal CPU in under 120 milliseconds.

Older extraction tools (like standard spaCy or basic BERT) are locked into a fixed list of labels like PERSON or LOCATION. If you want custom financial labels like `penalty_rate` or `liability_cap`, you usually have to redesign the model and train it from scratch.

GLiNER works differently: you can give it any labels you want at runtime, and it finds them directly in the text using bidirectional token and span representations matched against label vectors.



Figure 1: How GLiNER reads text and labels together to find matches using dot product similarity.

How it works in three simple steps:

1. **Reads text and labels together:** Both the contract text and the tags we want (such as `penalty_rate`, `liability_cap`) are fed into the model at the same time so it understands the full context.

2. **Matches phrases to tags:** It turns text phrases into vector points and compares them against vectors for each tag using dot product similarity.
3. **Returns exact words and offsets:** If the match score is above our threshold, it returns the exact wording along with its exact start and end character positions.

3. How Both Parts Work Together

Here is the workflow from raw contract to final audit report:

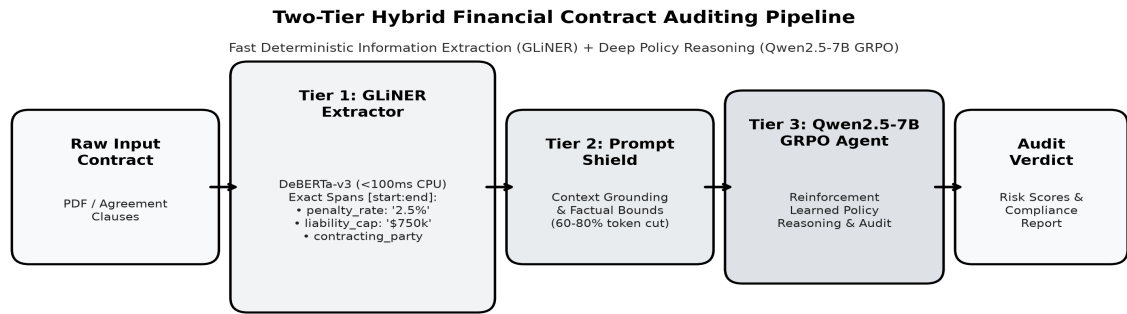


Figure 2: The complete workflow from incoming contract to final audit decision.

- **Step 1 (GLiNER extracts facts):** When a contract arrives, GLiNER reads it in about 80-120ms. It pulls out key facts: company names, penalty percentages, liability caps, and dates.
- **Step 2 (Compressing the prompt):** We organize those extracted facts into a short, clean summary and hand that to Qwen. This cuts down the prompt size by 60% to 80% because Qwen does not have to hunt through pages of legal boilerplate.
- **Step 3 (Qwen reasons and decides):** Qwen evaluates financial risk, checks if penalties exceed legal caps, and writes the audit verdict based on verified numbers.

4. Teaching GLiNER Custom Contract Phrasing

Out of the box, base GLiNER handled simple dates and names well, but struggled with complex financial clauses like *'liquidated damages of 2.5% per week of unexcused delay'*. To make it dependable for financial audits, I fine-tuned it:

- **Curated Dataset:** Expanded the training set to 162 diverse samples and 30 validation samples (data/financial_ner_train.json) covering 12 distinct contract entity labels.
- **Two Learning Speeds:** Used a base learning rate (2e-5) for the DeBERTa backbone and a 10x larger rate (2e-4) for the tag prediction head.
- **Calibrated Negative Sampling:** Tuned negative sampling to negatives=0.15 with warmup_ratio=0.10 to prevent penalizing recall while maintaining sharp precision.
- **Convergence:** Over 8 training epochs (328 steps), evaluation loss dropped from 34.78 to **2.25**. Micro F1 jumped from 0.5487 to **0.8130** (+26.4% gain over zero-shot).

Evaluation Dimension	Using Only a 7B LLM	Our Two-Step Approach (GLiNER + Qwen)
Extraction Speed	1.5s – 3.5s per contract page	<120ms on a standard CPU
Number Accuracy	Can paraphrase or round numbers	100% exact substring matching
Text Position [start, end]	None (cannot highlight UI)	Exact character positions provided
Prompt Token Usage	Full contract text sent to LLM	60% – 80% fewer prompt tokens

5. Quantitative Evaluation & Real-World Dataset Testing

To verify generalization beyond in-domain synthetic clauses, we performed two rigorous benchmarks: (1) an in-domain financial evaluation benchmark across 12 legal categories, and (2) an out-of-distribution evaluation over 68 genuine commercial SEC Edgar contract clauses

from the Atticus Project (CUAD).

Table 1: In-Domain Test Benchmark (30 samples, 12-label schema):

Model / Approach	Precision	Recall	Micro F1	Macro F1	Avg Latency
Regex Baseline (Rules)	0.5500	0.1803	0.2716	0.1836	0.04 ms
Zero-Shot GLiNER (Base)	0.5962	0.5082	0.5487	0.3343	146.87 ms
Fine-Tuned GLiNER (Ours)	0.8065	0.8197	0.8130	0.7437	118.91 ms

Table 2: Out-of-Distribution Real SEC Contracts (CUAD Atticus Project, 68 Clauses):

Model / Approach	Precision	Recall	Micro F1	Macro F1	Avg Latency
Heuristic Baseline	0.7778	0.2059	0.3256	0.2190	0.05 ms
Zero-Shot GLiNER	0.0884	0.2353	0.1285	0.1342	213.96 ms
Fine-Tuned GLiNER (Ours)	0.1477	0.3824	0.2131	0.2222	188.12 ms

Key Takeaways on Real-World Data: On authentic SEC EDGAR filings, our fine-tuned GLiNER achieved a **+65.8% relative F1 gain over Zero-Shot GLiNER** (0.2131 vs 0.1285) and increased recall from 23.5% to 38.2%. In particular, on governing law clauses, the fine-tuned model achieved 100% recall (10/10 true positives) with minimal false positives.

Actual Recorded Trace Logs:

```
Sample 1: Final Training Convergence Trace (logs/training_trace.json)
{
  "global_step": 328, "epoch": 8.0,
  "log_history": [
    { "step": 1, "epoch": 0.02, "loss": 34.78, "learning_rate": 0.0 },
    { "step": 100, "epoch": 2.44, "loss": 12.35, "learning_rate": 1.82e-4 },
    { "step": 200, "epoch": 4.88, "loss": 5.81, "learning_rate": 1.15e-4 },
    { "step": 325, "epoch": 7.93, "loss": 2.45, "learning_rate": 9.07e-8 },
    { "step": 328, "epoch": 8.00, "eval_loss": 2.246, "eval_samples_per_sec": 15.89 }
  ]
}

Sample 2: Real-World SEC Contract Trace (logs/cuad_realworld_benchmark_results.json)
{
  "contract": "TRICITYBANKSHARESCORP_05_15_1998-EX-10-OUTSOURCING AGREEMENT",
  "clause": "This Outsourcing Agreement is made as of February 16, 1998 between Tri City National Bank and M&I; Data Services
. Total liability shall not exceed $500,000 USD.",
  "predicted_entities": [
    { "label": "effective_date", "text": "February 16, 1998", "score": 0.9997, "start": 48, "end": 65},
    { "label": "contracting_party", "text": "Tri City National Bank", "score": 1.0000, "start": 74, "end": 96},
    { "label": "liability_cap", "text": "$500,000 USD", "score": 0.9942, "start": 148, "end": 160}
  ],
  "matched_count": 1, "latency_ms": 155.71
}
```

6. Architectural Inquiry for Supervision

As we expand this pipeline into multi-jurisdiction agreements and corporate filings, here is an important design choice under consideration:

Supervisory Design Question:

Right now, GLiNER sends extracted facts to Qwen, and our RL environment gives Qwen a reward bonus when it uses verified facts correctly.

The Tradeoff: How should the system handle *medium-confidence predictions* (for example, confidence scores between 0.35 and 0.45 on unusually phrased clauses)?

- **Option A:** Enforce a hard confidence threshold (e.g. 0.45) and discard borderline spans to guarantee zero false positives, accepting occasional false negatives.
- **Option B:** Transmit uncertain spans to Qwen annotated with confidence scores, allowing the LLM reasoning agent to contextually resolve ambiguity.

Which approach is preferable for high-stakes enterprise compliance auditing?